Vishwakarma Institute of Technology, Pune

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 6

Roll no:43 PRN No:12410889 Name: Akshada Ganesh Sadaphal

Subject Name: Deep Learning

Problem Statement: Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm:

1. **Start.**
2. Import TensorFlow, Keras, NumPy, Matplotlib and Scikit-learn libraries.
3. Specify the paths of the **training and validation datasets**.
4. Set the image size to **128 × 128 pixels**, batch size to **32**, and number of epochs to **20**.
5. Load the training images using image_dataset_from_directory().
6. Load the validation images using image_dataset_from_directory().
7. Extract and display the class names from the training dataset.
8. Apply TensorFlow prefetch() to improve data-loading performance.
9. Apply data augmentation using:
    - Random horizontal flipping
    - Random rotation
    - Random zoom
10. Build the CNN model.
11. Apply pixel normalization using Rescaling(1/255).
12. Add the first convolution layer with **32 filters**, followed by MaxPooling.
13. Add the second convolution layer with **64 filters**, followed by MaxPooling.
14. Add the third convolution layer with **128 filters**, followed by MaxPooling.

15. Flatten the extracted feature maps.
16. Add a Dense layer with **128 neurons** and ReLU activation.
17. Apply Dropout with a rate of **0.5** to reduce overfitting.
18. Add the output layer with the number of neurons equal to the number of disease classes and use **Softmax activation**.
19. Compile the model using:

- Optimizer: **Adam**
- Loss function: **Sparse Categorical Crossentropy**
- Metric: **Accuracy**

20. Train the CNN for **20 epochs** using the training and validation datasets.
21. Evaluate the trained model using the validation dataset.
22. Display validation loss and validation accuracy.
23. Save the trained model in .keras format.
24. Plot training and validation accuracy.
25. Plot training and validation loss.
26. Generate predictions for the validation dataset.
27. Generate the **classification report** containing precision, recall and F1-score.
28. Generate the **confusion matrix**.
29. **Stop.**

Code

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

# -------------------------------------------------
# 1. Dataset Path
# -------------------------------------------------
train_dir = r"D:\Coding\DL\ass6\tomato\train"
val_dir = r"D:\Coding\DL\ass6\tomato\val"

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
EPOCHS = 10

# -------------------------------------------------
# 2. Load Training Dataset
# -------------------------------------------------
train_dataset = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True
```

```
)

# ------------------------------------------------
# 3. Load Validation Dataset
# ------------------------------------------------
validation_dataset = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

# ------------------------------------------------
# 4. Class Names
# ------------------------------------------------
class_names = train_dataset.class_names
print("Classes:")

for i, name in enumerate(class_names):
    print(i, ":", name)
print("Number of classes:", len(class_names))

# ------------------------------------------------
# 5. Improve Data Loading
# ------------------------------------------------
```

```
AUTOTUNE = tf.data.AUTOTUNE
train_dataset = train_dataset.prefetch(
    buffer_size=AUTOTUNE
)
validation_dataset = validation_dataset.prefetch(
    buffer_size=AUTOTUNE
)

# ------------------------------------------------
# 6. Data Augmentation
# ------------------------------------------------
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])



# ------------------------------------------------
# 7. Build CNN Model
# ------------------------------------------------
model = models.Sequential([
    data_augmentation,

    # Normalize pixel values
```

```
layers.Rescaling(1.0 / 255),

# Convolution Block 1
layers.Conv2D(
    32,
    (3, 3),
    activation="relu",
    padding="same"
),

layers.MaxPooling2D((2, 2)),

# Convolution Block 2
layers.Conv2D(
    64,
    (3, 3),
    activation="relu",
    padding="same"
),

layers.MaxPooling2D((2, 2)),

# Convolution Block 3
layers.Conv2D(
    128,
```

```
        (3, 3),
        activation="relu",
        padding="same"
    ),

    layers.MaxPooling2D((2, 2)),

    # Flatten
    layers.Flatten(),

    # Fully Connected Layer
    layers.Dense(
        128,
        activation="relu"
    ),

    # Dropout
    layers.Dropout(0.5),

    # Output Layer
    layers.Dense(
        len(class_names),
        activation="softmax"
    )
])
```

```
# ------------------------------------------------
# 8. Build Model Before Summary
# ------------------------------------------------
model.build(input_shape=(None, 128, 128, 3))
model.summary()

# ------------------------------------------------
# 9. Compile Model
# ------------------------------------------------
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# ------------------------------------------------
# 10. Train Model
# ------------------------------------------------
history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=EPOCHS
)
```

```
# ------------------------------------------------
# 11. Evaluate Model
# ------------------------------------------------
loss, accuracy = model.evaluate(validation_dataset)
print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
print("Validation Accuracy (%):", accuracy * 100)

# ------------------------------------------------
# 12. Accuracy Graph
# ------------------------------------------------
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
```

```
plt.legend()
plt.grid(True)
plt.show()

# ------------------------------------------------
# 13. Loss Graph
# ------------------------------------------------
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

```
# ------------------------------------------------
# 14. Generate Predictions
# ------------------------------------------------
y_true = []
y_pred = []

for images, labels in validation_dataset:
    predictions = model.predict(
        images,
        verbose=0
    )

    predicted_classes = np.argmax(
        predictions,
        axis=1
    )

    y_true.extend(labels.numpy())
    y_pred.extend(predicted_classes)

# ------------------------------------------------
# 15. Classification Report
# ------------------------------------------------
print("\nClassification Report")
print("======================")
```

```
print(
    classification_report(
        y_true,
        y_pred,
        target_names=class_names
    )
)

# ------------------------------------------------
# 16. Confusion Matrix
# ------------------------------------------------
cm = confusion_matrix(
    y_true,
    y_pred
)

print("\nConfusion Matrix:")
print(cm)

# ------------------------------------------------
# 17. Display Confusion Matrix
# ------------------------------------------------
plt.figure(figsize=(10, 8))
```

```
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.colorbar()

plt.xticks(
    range(len(class_names)),
    class_names,
    rotation=90
)

plt.yticks(
    range(len(class_names)),
    class_names
)

plt.tight_layout()
plt.show()

# ----------------------------------------------
# 18. Test Sample Image
# ----------------------------------------------
```

```
for images, labels in validation_dataset.take(1):

    sample_image = images[0]
    actual_label = labels[0]

    prediction = model.predict(
        tf.expand_dims(sample_image, axis=0),
        verbose=0
    )

    predicted_label = np.argmax(prediction)
    confidence = np.max(prediction) * 100
    print("\nSample Prediction")
    print("-----------------")
    print("Actual Class:", class_names[actual_label])
    print("Predicted Class:", class_names[predicted_label])
    print("Confidence:", round(confidence, 2), "%")

# ------------------------------------------------
# 19. Save Model
# ------------------------------------------------
model.save(
    r"D:\Coding\DL\ass6\tomato_cnn_model.keras"
)
```

```
print("\nCNN model saved successfully!")


# ------------------------------------------------
# 20. Best Validation Accuracy
# ------------------------------------------------
best_val_accuracy = max(
    history.history["val_accuracy"]
)

best_epoch = np.argmax(
    history.history["val_accuracy"]
) + 1

print(
    "Best Validation Accuracy:",
    round(best_val_accuracy * 100, 2),
    "%"
)

print("Best Epoch:", best_epoch)
```

Output:

Output 1 — Dataset Loading

Found 10000 files belonging to 10 classes.

Training dataset loaded successfully!

Found 1000 files belonging to 10 classes.

Validation dataset loaded successfully!

Output 2 — Classes

Classes:

0 : Tomato___Bacterial_spot

1 : Tomato___Early_blight

2 : Tomato___Late_blight

3 : Tomato___Leaf_Mold

4 : Tomato___Septoria_leaf_spot

5 : Tomato___Spider_mites Two-spotted_spider_mite

6 : Tomato___Target_Spot

7 : Tomato___Tomato_Yellow_Leaf_Curl_Virus

8 : Tomato___Tomato_mosaic_virus

9 : Tomato___healthy

Number of classes: 10

Output 3 — Sample Dataset Images

```
import zipfile
import os

zip_path = r"D:\Coding\DL\ass6\ass6.zip"
extract_path = r"D:\Coding\DL\ass6"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully!")
```
[9]

```
Dataset extracted successfully!
```

```
import os

base_path = r"D:\Coding\DL\ass6"

print(os.listdir(base_path))
```
[10]

```
['ass6.ipynb', 'ass6.zip', 'cnn_tomato.py', 'tomato']
```

```
import os

base_path = r"D:\Coding\DL\ass6"

for root, dirs, files in os.walk(base_path):
    if "train" in dirs and "val" in dirs:
        print("Dataset location:")
        print(root)
```
[11]

```
Dataset location:
D:\Coding\DL\ass6\tomato
```

```
train_dir = r"D:\Coding\DL\ass6\tomato\train"
val_dir = r"D:\Coding\DL\ass6\tomato\val"

print("Train directory:", train_dir)
print("Validation directory:", val_dir)
```
[13]

```
Train directory: D:\Coding\DL\ass6\tomato\train
Validation directory: D:\Coding\DL\ass6\tomato\val
```

```
import os

print("Train path exists:", os.path.exists(train_dir))
print("Validation path exists:", os.path.exists(val_dir))
```
[14]

```
Train path exists: True
Validation path exists: True
```

```
print("Training folders:")
print(os.listdir(train_dir))
```
[15]

```
Training folders:
['Tomato__Bacterial_spot', 'Tomato__Early_blight', 'Tomato__healthy
```

```
train_dataset = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True
)

print("Training dataset loaded successfully!")
```
[16]

```
Found 10000 files belonging to 10 classes.
Training dataset loaded successfully!
```

```
validation_dataset = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

print("Validation dataset loaded successfully!")
```
[17]

```
Found 1000 files belonging to 10 classes.
Validation dataset loaded successfully!
```

```python
class_names = train_dataset.class_names

print("Classes:")

for i, name in enumerate(class_names):
    print(i, ":", name)

print("\nNumber of classes:", len(class_names))
```

[18]

```
Classes:
0 : Tomato___Bacterial_spot
1 : Tomato___Early_blight
2 : Tomato___Late_blight
3 : Tomato___Leaf_Mold
4 : Tomato___Septoria_leaf_spot
5 : Tomato___Spider_mites Two-spotted_spider_mite
6 : Tomato___Target_Spot
7 : Tomato___Tomato_Yellow_Leaf_Curl_Virus
8 : Tomato___Tomato_mosaic_virus
9 : Tomato___healthy

Number of classes: 10
```

```python
plt.figure(figsize=(10, 10))

for images, labels in train_dataset.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

plt.tight_layout()
plt.show()
```

[19]

